The Even-Index Encoding for Rectilinear Polygons: Simplicity, Run-Time, and Memory

Wai-Shing Luk

***Abstract*—Rectilinear polygons are the native geometry of VLSI physical design. The conventional representation stores all n boundary vertices; because edges alternate horizontal and vertical, the x-first even-index encoding stores only $m = n/2$ points and recovers the rest by inserting (x_{i+1}, y_i). We compare the representations on simplicity, run time, and memory. It halves storage ($2.00\times$) and outer-loop iterations with unchanged asymptotic complexity; a C++20 benchmark measures a $2.00\times$ bounding-box speedup, but only about $1.3\times$ for area and canonicality, while an $O(m)$ expansion costs 7–$13\times$ the full-vertex area pass. A full-vertex convex decomposition is valid but $12\times$ slower at $n = 260$.**

I. Introduction

A modern integrated circuit is a physical object with billions of components, and physical design turns an abstract netlist into the polygons that a foundry can fabricate [1], [2], [3]. The geometry of the layout is overwhelmingly *rectilinear*: every edge is horizontal or vertical, and every coordinate is an integer. A 45° edge is manufacturable, but the CAD/EDA tools and the algorithms that manipulate the layout are built around axis-aligned geometry; rectilinearity is the practical representation. Rounding is not merely imprecise but illegal, because two edges that should be collinear may miss and a design-rule check fails. At the same time, the individual objects are small: a standard-cell polygon has at most about one hundred vertices, a net a handful of pins, and a block a score of metal layers. At these sizes a quadratic algorithm is not merely acceptable: with a small constant it can be faster than a sub-quadratic method whose overhead dominates. Exactness, simplicity, and verifiability matter more than the asymptotic exponent.

This paper is about the *representation* of a rectilinear polygon and the algorithms that consume it. The usual representation is the full boundary, a cyclic list of all n vertices. Because consecutive edges alternate between horizontal and vertical, every other vertex determines the omitted one under a suitable convention. The resulting *even-index* representation stores only $m = n/2$ points. The question we ask is not whether the encoding is valid—it is—but what it actually buys: which advantages are inherent to the representation, which are consequences of a particular implementation, and what it costs. The short answer is that the encoding halves storage and the number of outer-loop iterations, leaves the asymptotic order unchanged, and yields a full $2\times$ speedup only where the per-iteration work is unchanged; area and canonicality improve by less than $2\times$, and a workload that needs the full boundary on every operation pays an $O(m)$ expansion that can erase the gain. The expansion convention must be fixed, and a stored list can be non-canonical if a shared-axis point lies between its neighbours or if cyclic consecutive points coincide. The full boundary is recovered in $O(m)$ by the x-first rule. GDSII files store rectilinear polygons as arbitrary polygons, so halving the stored vertices is a direct saving. Thus n is the number of boundary vertices and $m = n/2$ the number of stored points.

The input throughout this paper is a **simple rectilinear polygon** P, and S is its even-index encoding. For such an encoding, cyclic consecutive stored points are distinct and differ in both coordinates; every simple-polygon encoding is therefore canonical. Canonicality is a local property of the encoding: it does not imply simplicity, and a canonical list can repeat a non-adjacent point or expand to a self-intersecting boundary. Simplicity remains an assumption on the input, checked separately where the implementation needs a guard; the cut and canonicalization operations preserve it. The proofs below therefore work directly with simple input polygons and their encodings.

A canonical form is a fixed normalization rather than an extra geometric assumption: by standardizing the traversal (x-first, even-index), the algorithms avoid repeated ad-hoc checks. The implementation shares one direction-key template between the axes; the only asymmetry is the x-first expansion rule, which a coordinate swap handles for the y variant. Canonical form is often treated as a convention rather than a result. We state it explicitly because the boundary cases (two-point lists, consecutive duplicates, and non-adjacent repeats) are easy to get wrong in an implementation.

We make the following contributions.

1. We define the x-first even-index encoding and its canonical form, give the reconstruction rule, the compressed perimeter, and the area formulas (Section III).
2. We separate inherent from implementation-dependent advantages and give an exact cost model for storage, passes, and expansion (Section IV).
3. We measure both representations in C++20 on staircase polygons from $m = 128$ to 8192: the byte ratio reaches $2.00\times$, the bounding box is 1.93–$2.00\times$ faster, area is about $1.3\times$ faster at the largest sizes, canonicality is about $1.3\times$ faster, and expansion costs 7–$13\times$ the full-vertex area pass at $m = 8192$

(Section XI).

4. We implement the convex decomposition on the full vertex list as a baseline; it preserves area and convexity, and we compare its code size, run time, and memory with the even-index version. It runs 2–12× slower on the same fixtures, with the gap growing with size (Section XI).
5. We show that the encoding makes the small base cases explicit: two stored points are a rectangle and three are an orthogonal L-shape, so the cut recursion terminates on primitives rather than on boundary vertices (Section VI).
6. We give a recursive cut framework and a three-step rectangle decomposition in $O(m^2)$, and prove that the output is a partition into rectangles (Section VII, Section IX).
7. We state when the full-vertex list remains the better choice: dynamic vertex insertion, T-junctions, non-rectilinear geometry, and weakly simple boundaries with bridges (Section XII).

The terms used in these statements are defined before they are used: the representation and canonical form in Section III; the cost model in Section IV; the canonicalization and hull operations in Section V; and the cut operations and their helper functions in Section VI.

The remainder of the paper is organized as follows. Section II reviews related work. Section III fixes notation, states the canonical form, and gives the reconstruction rule. Section IV compares the two representations on simplicity, run time, and memory. Section V presents creation, canonicalization, and monotone hulls. Section VI develops the recursive cut framework. Section VII develops convex decomposition, and Section IX develops the three-step rectangle stage. Section XI reports the implementation, the micro-benchmark, and the tests, Section XII discusses scope and threats to validity, and Section XIII concludes.

II. Related Work

Rectilinear polygons are a classical topic in computational geometry and a workhorse of VLSI layout. The textbook treatment covers monotone polygons, convex decomposition, and rectilinear convex hulls [4], [5], [6]. The VLSI literature uses the same primitives for layout fracturing and for decomposing L-shaped obstacles into rectangles; double-patterning and resolution-enhancement flows make rectangle decomposition a manufacturing requirement rather than a convenience [7], [8].

The optimization literature on rectangle and convex decomposition is extensive. Ohtsuki [9] gave an $O(n^{5/2})$ algorithm for minimum dissection of rectilinear regions. Lipski [10] developed a Manhattan-path approach that yields an $O(n^{3/2} \log n)$ minimum dissection, and Nahar and Sahni [11] gave a fast horizontal-cut decomposition. Soltan and Gorpinevich [12] gave an $O(n^{3/2} \log n)$ algorithm for polygons with arbitrary holes. For convex decomposition, Keil [13] gave a polynomial dynamic-programming algorithm for simple polygons, while Lingas [14] showed that minimum convex partition becomes NP-hard in the presence of holes. Keil's survey [6] and the textbook of de Berg et al. [5] provide the standard background.

Compact layout representations are also classical. Corner stitching [15] stores rectangles and their neighbors rather than a full polygon boundary. Industrial stream formats store one integer coordinate pair per vertex: the GDSII `XY` record is an array of 4-byte signed integers [16], and OASIS adds implicit-Manhattan point lists that store only alternating edge deltas and leave the final two displacements implicit [17]. The even-index encoding removes the same alternation redundancy, but at the vertex level: it keeps every other vertex, so the omitted coordinate is recovered by the x-first rule rather than by an edge delta. Franklin and Magalhães [18] show that a vertex set alone does not determine an axis-aligned polygon and that one extra bit per vertex restores uniqueness; our canonical form is a different normalization, in which the parity and the x-first convention replace that bit. The representation is therefore not a new geometric object. Its interest is the exact accounting of what the halving buys.

To our knowledge, the specific even-index encoding and its measured comparison with the full-vertex list have not been isolated in the literature; the ingredients—vertex/edge alternation, implicit vertices, and per-vertex coordinate storage—are standard. We therefore frame the contribution as a formal reconstruction rule, an exact cost model, and a reproducible measurement, not as the invention of the encoding.

Exact integer geometry avoids the rounding errors of floating-point predicates. Symbolic perturbation [19] is a complementary technique for degeneracies; exact integer arithmetic avoids rounding but does not by itself remove duplicate vertices, zero-length edges, or collinear points. Our algorithms handle those cases through the canonicality predicate and the canonicalization pass.

The present work differs from the optimization literature in two ways. First, it focuses on a compressed even-index representation and on the implementation of the cut primitives on that representation. Second, it develops a rectangle decomposition on top of the convex decomposition, so that the output is a set of rectangles rather than only convex pieces. The pipeline is deliberately simple; it makes no optimality claim and is intended as a reliable preprocessing step.

III. Preliminaries and Representation

A. Rectilinear Polygons and Orthogonal Convexity

A *rectilinear* (or *orthogonal*) polygon is a simple closed polygon whose edges are axis-aligned. Its vertices alternate between horizontal and vertical edges, so it has an even number of vertices. For a polygon with holes, make a cut (a *bridge*) from the outer boundary to each hole. A zero-width bridge is traversed in both directions, so the

area is unchanged and the boundary is *weakly simple* (a narrow bridge gives a simply connected polygon). The even-index encoding applies to this boundary; repeated bridge vertices are handled by the degenerate split of Section VI. This reduction is standard and is not specific to the even-index representation. We state the proofs for a simply connected input; the bridge reduction supplies that case. We also assume the full boundary is reduced, i.e. it has no 180° vertices; the canonicalization pass of Section V removes consecutive duplicates and redundant collinear points before the representation is used. Rather than assuming that the input has no redundant points, we state the reduced-boundary assumption and canonicalize the remaining redundancies.

A rectilinear polygon is *orthogonally convex* (or *rectilinearly convex*) if every horizontal and every vertical line intersects it in a connected set, or in the empty set. Equivalently, it is both x-monotone and y-monotone. Orthogonal convexity is not Euclidean convexity: an L-shape is orthogonally convex, although its interior angle at the inner corner is 270°. We use "convex" only in the orthogonal sense in this paper and write "orthogonally convex" when the distinction matters.

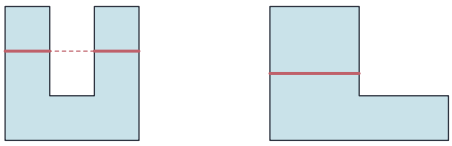


Figure 1: Left: an L-shape is both x- and y-monotone and is therefore orthogonally convex. Right: a U-shape is x-monotone but a horizontal line meets it in two intervals, so it is not orthogonally convex.

B. Even-Index Encoding

Let $v_0, \ldots, v_{n-1}$ be the vertices of a simple rectilinear polygon in boundary order, with n even. Rotate the list so that the edge from v_0 to v_1 is horizontal. The *even-index encoding* stores

$$S = (s_0, \ldots, s_{m-1}) = (v_0, v_2, \ldots, v_{n-2}), \qquad m = n/2. \tag{1}$$

The omitted odd-index vertices are implicit. With the x-first convention used throughout this paper, the implicit vertex between $s_i = (x_0, y_0)$ and $s_{i+1} = (x_1, y_1)$ is

$$I_i = (x_1, y_0), \tag{2}$$

inserted before s_{i+1}, including the closing pair $s_{m-1} \to s_0$. The resulting cyclic sequence is $E(S)$. The diagonal chord $s_i s_{i+1}$ is not a boundary edge; the boundary follows the horizontal edge to I_i and then the vertical edge to s_{i+1}. The y-first form (x_0, y_1) reconstructs a different polygon and is not used.

For an encoding of a simple polygon, every cyclic consecutive pair of stored points differs in both coordinates and no two cyclic consecutive points coincide, because the omitted vertex between them is a genuine corner. A two-point representation is a rectangle when the points differ in both coordinates; a distinct two-point list that shares an axis is a degenerate segment that is canonical but not a simple polygon. A three-point representation is an L-shaped monotone piece when the expansion is simple; more points can describe a larger polygon, and orthogonal convexity is checked separately on the expansion. The stored order follows the boundary, and the full boundary is reconstructed on demand in $O(m)$ time. The implementation stores the first stored point as an origin and the remaining points as displacement vectors, so translating the polygon updates the origin in $O(1)$. Figure Fig. 3 shows stored and implicit vertices on an eight-vertex U-shape.

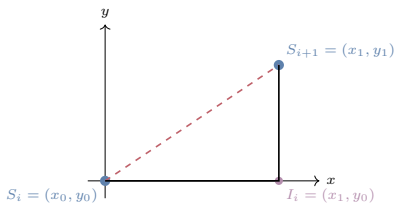


Figure 2: The x-first expansion inserts the implicit vertex $I_i = (x_1, y_0)$ between S_i and S_{i+1}. The dashed chord is not a boundary edge.

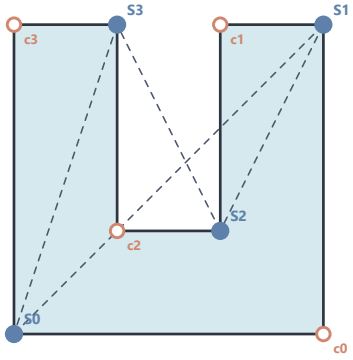


Figure 3: Stored (filled) and implicit (hollow) vertices of an eight-vertex U-shape. Dashed segments connect consecutive stored points; they are not polygon edges. The compressed representation has four stored points.

C. Canonical Form

Definition 1 (canonical form). A stored sequence $S = (s_0, \ldots, s_{m-1})$ is *canonical* if

1. $m \geq 2$;
2. no two cyclically consecutive points coincide:

$$s_i \neq s_{i+1} \quad \text{(indices modulo } m\text{)}; \tag{3}$$

3. if $m = 2$, the two points are distinct; they may share an x or y coordinate;

Algorithm 1 IsCanonical(S)

Require: stored points $S = (s_0, \ldots, s_{m-1})$
if $m < 2$ **then**
  return False
end if
for $i \leftarrow 0$ **to** $m-1$ **do**
  $j \leftarrow (i+1) \bmod m$
  if $s_i = s_j$ **then**
    return False
  end if
end for
if $m = 2$ **then**
  return True
end if
for $i \leftarrow 0$ **to** $m-1$ **do**
  $p \leftarrow (i-1) \bmod m$; $q \leftarrow (i+1) \bmod m$
  if $y_p = y_i$ **and** x_i strictly between x_p and x_q **then**
    return False
  end if
  if $x_i = x_q$ **and** y_i strictly between y_p and y_q **then**
    return False
  end if
end for
return True

4. if $m \geq 3$, no stored point is *redundant*: for every i, with $p = s_{i-1}$ and $q = s_{i+1}$,

$$\neg\big(y_p = y_i \wedge x_i \text{ strictly between } x_p, x_q\big) \quad \text{and} \quad \neg\big(x_i = x_q \wedge y_i \text{ strictly between } y_p, y_q\big). \tag{4}$$

A stored point is *redundant* when deleting it and re-expanding the remaining points preserves both the perimeter and the area. The two tests in (4) are exactly the deletable cases: if p and s_i share a y coordinate and x_i lies between x_p and x_q, deleting s_i merges two collinear horizontal edges; if s_i and q share an x coordinate and y_i lies between y_p and y_q, it merges two collinear vertical edges. In both cases the represented boundary is unchanged, so the perimeter and the area are preserved. If neither test holds, deleting s_i changes the perimeter or the area. We call a shared-axis point that is not between its neighbours a *spike*. A spike is a zero-width protrusion: deleting it would shorten the boundary, so it is not redundant.

For $m = 2$ the expansion has coincident vertices; we identify them, so the two stored points are the only distinct vertices. They are the opposite corners of a rectangle; when they are distinct, deleting one leaves a one-point list and changes the perimeter, while two coincident points are a duplicate and are rejected by condition 2.

The condition is structural: it constrains the encoding, not the geometry. A canonical list may repeat a non-adjacent point or expand to a self-intersecting boundary. Algorithm 1 checks it directly. The quantity $m < 2$ is rejected, because a one-point list would expand to a single repeated vertex and an empty list does not describe a polygon at all.

The check is a single $O(m)$ pass: the consecutive-duplicate scan and the betweenness scan are each $O(m)$, and no sorting or all-pairs scan is needed.

Lemma 1 (the encoding is canonical). If P is a simple rectilinear polygon and S is its even-index encoding, then S is canonical.

Proof. The boundary of a simple rectilinear polygon has at least four vertices, so $m = n/2 \geq 2$. The vertices are distinct, hence no two cyclically consecutive stored points coincide. Between two consecutive stored points s_i and s_{i+1}, the omitted vertex I_i is a genuine corner: the edge from s_i to I_i is horizontal and the edge from I_i to s_{i+1} is vertical, so $x_i \neq x_{i+1}$ and $y_i \neq y_{i+1}$. The closing pair is handled by the same argument. □

Table I lists the canonical-form examples and test cases. The row $((0,0),(1,0),(2,1))$ is the concrete reason for checking canonicality: its area and perimeter equal those of the two-point list $((0,0),(2,1))$, but the middle point is redundant. An area-only or perimeter-only test cannot detect the redundant encoding; the canonicality check rejects it.

D. Perimeter and Area

The perimeter of the represented polygon is

$$P(S) = \sum_{i=0}^{m-1} \big(|x_{i+1} - x_i| + |y_{i+1} - y_i|\big), \tag{5}$$

with indices modulo m. Deleting a redundant stored point preserves both $P(S)$ and the area by definition; the betweenness test of Section V is the local criterion.

The signed area, in the form used by the implementation, is

$$A(S) = v_1^x v_1^y + \sum_{i=2}^{m-1} v_i^x \big(v_i^y - v_{i-1}^y\big), \qquad v_i = s_i - s_0, \tag{6}$$

where $s_0, \ldots, s_{m-1}$ are the stored points. For a two-point list the formula reduces to $(x_1 - x_0)(y_1 - y_0)$; this is a rectangle when both coordinates differ and zero for a degenerate segment. The sign is positive for a counter-clockwise polygon and negative for a clockwise one; the orientation flag carries this sign through all algorithms.

Lemma 2 (area). For a simple rectilinear polygon, the formula (6) equals the signed area of the reconstructed boundary.

Proof sketch. Expand the signed-area sum over the reconstructed sequence $s_0, I_0, s_1, \ldots$. Each implicit vertex contributes terms that cancel with the adjacent stored vertices, leaving the telescoping expression above. A direct expansion for the three-point L-shape and the two-point list verifies the base cases; the general case follows by induction on m. □

E. Base Cases

The compressed representation turns the two smallest decompositions into base cases: for a simple input polygon, two stored points are a non-degenerate rectangle, and three stored points are an L-shaped monotone piece when the expansion is simple. These are not degenerate cases; they expand to four and six boundary vertices, respectively.

Table I: Canonical-form examples and test cases.

Stored points	Canonical	Reason
$(2,1),(2,0)$	yes	two distinct points; shared axis allowed at $m=2$
$(2,1),(3,-3)$	yes	rectangle corners
$(2,1),(2,0),(3,-3)$	yes	spike; shared x, point not between
$(0,0),(0,0)$	no	cyclic consecutive duplicate
$(0,0),(0,1),(0,2)$	no	middle point between
$(0,0),(1,0),(2,0)$	no	middle point between
$(0,0),(1,0),(2,1)$	no	middle point between
$(0,0),(1,1)$	yes	rectangle
$(0,0),(1,1),(0,0),(2,2)$	yes	non-adjacent duplicate; canonical, not simple
$(0,2),(4,0),(2,4)$	yes	canonical, not simple
$(0,0),(2,0),(1,1)$	yes	spike; shared y, point not between
$(0,0)$	no	fewer than two points

Canonicality alone also admits a distinct two-point list that shares an axis; it expands to a degenerate segment, is not a simple polygon, and is not used as a decomposition base case. Since the input polygon is simple, both base cases are simple and canonical by Lemma 1.

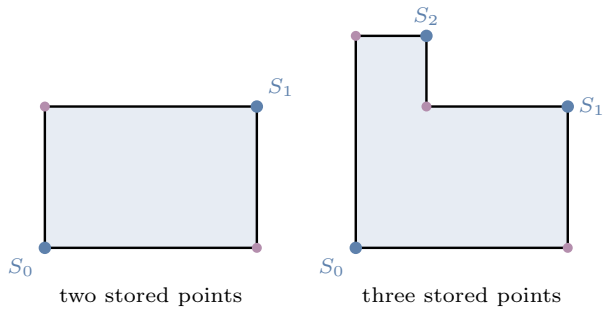


Figure 4: The two smallest encodings: two stored points (opposite corners of a rectangle) and three stored points (an L-shaped monotone piece).

IV. Representation Trade-offs

A. Simplicity

Table II compares the two representations as data structures. The saving in records is only the beginning: every pass that ranges over *corners* ranges over m units instead of n, and the two smallest decomposition cases are two and three stored points rather than four and six boundary vertices. In the cut framework this removes the need to special-case a four-vertex rectangle and a six-vertex L-shape; the recursion terminates on the representation's own primitives (Section VI). Both representations admit a direction-key template; the only axis asymmetry is the x-first expansion rule, which a coordinate swap handles. The creation, hull, and cut routines are therefore written once and instantiated for x and y in both cases.

Code size is a separate question. We count non-blank, non-comment lines (NBNC) of the two convex-decomposition implementations; the even-index count includes the convex-specific functions and the two shared helpers they call (`_find_min_dist_point` and `try_degenerate_two_node_split`), and the full-vertex count is the function bodies of `rpolygon_cut_full.cpp`. Table III gives the result. The decomposition logic is almost identical (185 versus 187 NBNC), but the even-index canonicalizer is 83 NBNC against 33 for the full-vertex module, so a canonical-output pipeline is 268 versus 220. The even-index version's smaller base cases are offset by the compressed predicate, the degenerate split, and a more elaborate canonicalizer. The line count therefore does not show that the encoding is smaller in source; its measured advantage is memory and run time.

The price is a convention that must be fixed: the parity (start on a horizontal edge) and the orientation convention. Canonicality is a *local* property—it constrains cyclic consecutive stored points, not the whole boundary—and it is checked in one $O(m)$ pass. It is not a simplicity certificate: a canonical list can repeat a non-adjacent point or expand to a self-intersecting boundary. The encoding is therefore simple to check but delicate to construct, and every producer of stored points must respect the x-first convention. The full-vertex list has no such convention; any vertex order works, which is why it remains the format of interchange.

B. Run-Time

Two statements must be separated. The first is a counting statement: any pass whose outer loop ranges over the stored units performs exactly m iterations instead of n, a factor of two. The second is a timing statement, and it holds only when the per-iteration work is unchanged. Table IV gives the counts and the measured ratios.

One fairness point about the area row. Both sides call the repository's current telescoping sum (`RPolygon::signed_area` and `Polygon::signed_area_x2`), which performs one product per stored offset; the only difference is whether the offsets come from m stored points or from n boundary vertices. The measured area ratio therefore isolates the representation, not the formula.

The bounding box is the clean case: the implicit vertex $I_i = (x_{i+1}, y_i)$ reuses a stored x and a stored y, so the extremal coordinates of the polygon are exactly those of the stored points. The per-iteration work is identical and the measured ratio is 2.00. Area is the instructive case: both classes use a telescoping formula of the same shape,

Table II: Structural comparison of the full-vertex list and the x-first even-index encoding for a reduced simple rectilinear polygon with n vertices and $m = n/2$ stored points.

Aspect	Full vertex list	Even-index encoding
Stored records	n points	m points
Omitted vertices	none	m, recovered by one rule
Canonicality check	duplicate and collinear scan over n	local betweenness scan over m
Rectangle base case	4 vertices	2 points
L-shape base case	6 vertices	3 points
Axis convention	none	fixed x-first; y variant by swap
Interchange with GDSII	direct	expand first

Table III: Non-blank, non-comment source lines (NBNC) for the convex decomposition (C++20). The decomposition rows count function bodies; canonicalization is separate. The last row is context only: the even-index file also implements the explicit, implicit, and rectangle policies.

Scope	Even-index	Full-vertex
Decomposition logic	185	187
Canonicalization	83	33
Total, canonical output	268	220
Whole cut module (context)	495	242

Table IV: Cost model and measured ratio. $n = 2m$. "Trip" is the outer-loop count; "measured" is the C++20 benchmark of Section XI (median of seven trials).

Pass	Full list	Even-index	Trip ratio	Measured
Storage records	n	m	2	2.00× bytes
Bounding box / extremal	n	m	2	1.93–2.00×
Area	$n-1$ terms	$m-1$ terms	≈ 2	$\approx 1.3\times$ (large m)
Canonicality	n tests	m tests	2	about 1.3×
Perimeter	n edges	m pairs, two legs each	1 arithmetic	not measured
Expansion	—	$O(m)$	—	7–13× full area
Cut	$O(n^2)$ pairs	$O(m^2)$ pairs	4 pair count	not measured

but the stored loop copies the current vector while the full loop indexes three entries, so the per-iteration constants differ and the measured ratio is about 1.3 at the largest sizes rather than 2 (at smaller sizes it varies between 1.1 and 1.7 as the constant terms dominate). Perimeter has no arithmetic saving at all: the compressed formula sums $|dx| + |dy|$ over stored pairs, which is exactly the two legs between consecutive stored points, so the number of absolute values and additions is the same as over n edges. Only loop overhead halves. Canonicality compares two different predicates (betweenness on stored points versus collinearity on full vertices), so its measured ratio of about 1.3 is not a pure representation ratio.

Expansion is the cost that is easy to forget. Materializing the full boundary is $O(m)$ and, at $m = 8192$, takes 138–183 μs across runs, while the full-vertex area pass takes 14–19 μs: expanding before a single pass is 7–13 times more expensive than storing the full list and running that pass. If a workload expands on every operation, the compressed representation is a net loss; the gain exists only when the passes can be formulated over stored points, and when several such passes amortize one expansion or avoid it. The cut stage is $O(m^2)$ instead of $O(n^2)$, but the pair count falls by four while each stored unit touches two edges, so we make no measured speedup claim for it.

C. Memory

In the C++ implementation a coordinate is a 32-bit integer, so a point is two integers and eight bytes; both classes add a 32-byte header for the origin and the vector. With n vertices,

$$B_{\text{even}}(n) = 32 + \left(\frac{n}{2} - 1\right) 8, \qquad B_{\text{full}}(n) = 32 + (n-1)8, \tag{7}$$

so the ratio $B_{\text{full}}/B_{\text{even}}$ tends to two and is already 1.91 at $n = 64$ and 2.00 at $n = 4096$ (Table V). The measured allocations agree with the model.

The same counting applies to interchange. A GDSII `XY` record stores one 8-byte coordinate pair per boundary vertex [16], so the boundary payload is halved; OASIS already removes the same redundancy at the edge level by storing alternating deltas [17]. The saving is in steady-state storage, not necessarily in peak memory: an expansion allocates n points temporarily. In a Python implementation each point is a 48-byte object plus an 8-byte list slot, so the absolute numbers are larger, but the ratio still approaches two because the object count halves. The memory advantage is therefore the most robust of the three: it follows from the record count and does not depend on the per-iteration constants.

V. Creation, Canonicalization, and Monotone Hulls

A. Creation

The creation pass takes a point set and returns an ordering of the points that is intended to be a monotone chain. For an x-monotone ordering, sort the points by (x, y) and split them into a chain below the leftmost point and a chain

Table V: Storage: byte model and measured allocation (C++20, `Point<int>` and `Vector2<int>` are 8 bytes each).

n	Even-index bytes	Full bytes	Ratio	Measured ratio
64	280	536	1.91	1.91
256	1048	2072	1.98	1.98
1024	4120	8216	1.99	1.99
4096	16408	32792	2.00	2.00
16384	65560	131096	2.00	2.00

Algorithm 2 CreateMonotone(P, dir, cmp)

Require: point set P, direction key dir, comparison cmp
Ensure: an ordering of P
1: $p_{\min} \leftarrow \arg\min_{p \in P} dir(p)$; $p_{\max} \leftarrow \arg\max_{p \in P} dir(p)$
2: split P into P_1 and P_2 according to the side of $p_{\min}$
3: sort P_1 by dir; sort P_2 by dir in reverse
4: **return** P_1 concatenated with P_2

above it; concatenate the lower chain with the reversed upper chain. The y-monotone version uses (y, x) as the sort key. Algorithm 2 states the shared template.

The split is deterministic. Write $dir(p) = (dir_1(p), dir_2(p))$. If the caller's comparison selects the lower chain, then

$$P_1 = \{p \in P : dir_2(p) \le dir_2(p_{\min})\}; \tag{8}$$

otherwise the inequality is reversed. In both cases $P_2 = P \setminus P_1$. The algorithm sorts P_1 ascending by dir and P_2 descending, and returns the concatenation $P_1 P_2$. This is exactly the ordering used by the implementation; it is a chain order, not yet a polygon.

The creation pass returns an *ordering*, not necessarily a canonical list. Figure Fig. 5 shows the two orderings produced by the x-monotone and y-monotone creation routines on the twelve-point fixture used by the test suite. Both orderings contain consecutive collinear triples and therefore fail canonicality. Canonicalization removes the redundant collinear points from each ordering; the result may still be non-simple.

B. Monotone Hull

The input to the hull pass is a stored sequence S and an orientation flag $\sigma \in \{\text{CLOCKWISE}, \text{ANTICLOCKWISE}\}$. The hull pass computes a monotone hull. It finds the minimum and maximum vertices in the sweep direction, builds a circular doubly linked list, and walks the two chains from minimum to maximum and back. At a vertex with previous, current, and next points p, c, n, it computes

$$\Delta = (y_c - y_p)(x_n - x_c). \tag{9}$$

A vertex is removed when the sweep coordinate violates monotonicity and Δ has the wrong sign for the orientation. Removal is $O(1)$ in the linked list. Algorithm 3 states the procedure.

Proposition 1 (termination and cost). Algorithm 3 terminates. Each sweep is $O(m)$. The collinear scan used by the monotone-hull implementation is $O(m^2)$ in the worst case.

Algorithm 3 MonotoneHull(S, σ, dir)

Require: stored points S, orientation σ, sweep direction dir
Ensure: a monotone hull
1: $v_{\min} \leftarrow \arg\min_{v \in S} dir(v)$; $v_{\max} \leftarrow \arg\max_{v \in S} dir(v)$
2: build a circular linked list over S
3: **for** each chain $v_{\min} \to v_{\max}$ and $v_{\max} \to v_{\min}$ **do**
4: $\quad c \leftarrow$ first vertex of the chain
5: $\quad$ **while** $c \neq$ stop vertex **do**
6: $\quad\quad p \leftarrow prev(c)$; $n \leftarrow next(c)$
7: $\quad\quad$ **if** $dir(c) > dir(n)$ **or** $dir(p) > dir(c)$ **then**
8: $\quad\quad\quad \Delta \leftarrow (y_c - y_p)(x_n - x_c)$
9: $\quad\quad\quad$ **if** Δ has the wrong sign for σ **then**
10: $\quad\quad\quad\quad$ detach c; $c \leftarrow p$
11: $\quad\quad\quad$ **else**
12: $\quad\quad\quad\quad c \leftarrow n$
13: $\quad\quad\quad$ **end if**
14: $\quad\quad$ **else**
15: $\quad\quad\quad c \leftarrow n$
16: $\quad\quad$ **end if**
17: $\quad$ **end while**
18: **end for**
19: remove collinear stored points from S
20: **return** S

Proof. Every detach strictly decreases the number of stored vertices, so the loop terminates. Each vertex is visited $O(1)$ times per chain and each detach is $O(1)$, hence $O(m)$ per sweep. The collinear cleanup may remove $O(m)$ points and restart an $O(m)$ scan after each removal, hence $O(m^2)$ worst case. □

The loop stops only when no stored vertex violates the monotonicity condition, so each surviving chain is monotone at termination. The proposition therefore gives termination, monotonicity, and the stated cost; no minimality claim is needed.

C. Canonicalization: Removing Redundant Vertices

Canonicalization applies to a raw creation ordering. An ordering can still contain redundant stored points. In the x-first encoding, two cyclic consecutive points with the same y make the second point lie on a horizontal edge, while two with the same x make the first point lie on a vertical edge. Removing such a point preserves the represented boundary only when the point lies *strictly between* its neighbours. Algorithm 4 enforces

that condition. It removes cyclic consecutive duplicates first, then uses a work queue so that only the neighbours of a removed point are re-examined.

Lemma 3 (canonicalization preserves perimeter and area). If a removed stored point lies strictly between its neighbours, deleting it preserves the represented boundary, hence both the perimeter and the signed area. Algorithm

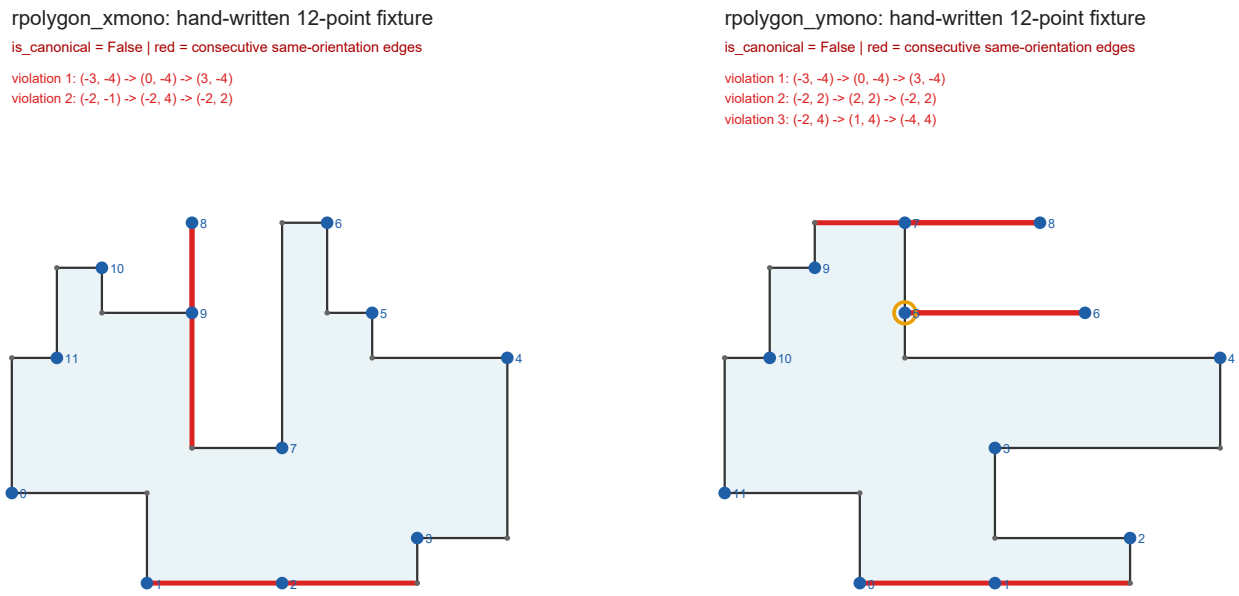


Figure 5: Non-canonical outputs of the creation routines on the twelve-point fixture: (left) the x-monotone ordering; (right) the y-monotone ordering. Red segments are consecutive collinear edges. Both lists fail canonicality.

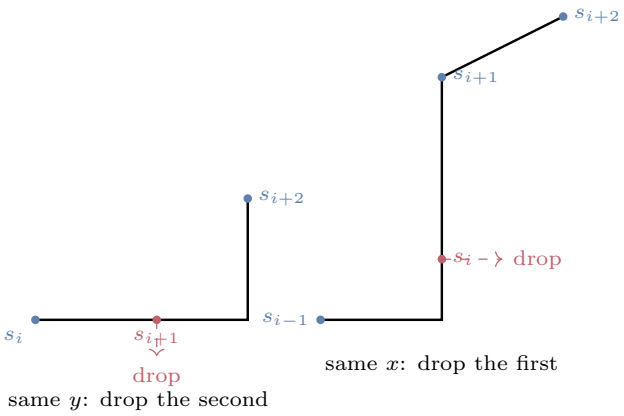


Figure 6: Betweenness removal: a same-y pair drops the second point; a same-x pair drops the first.

Algorithm 4 CANONICALIZE(S)

Require: stored points $S = (s_0, \ldots, s_{m-1})$
Ensure: a canonical list, or a failure if an unexpected obstruction remains
remove cyclic consecutive duplicates from S
if $|S| \le 2$ **then**
  return S
end if
build a circular doubly linked list over S and a queue of all indices
while the queue is nonempty **and** $|S| > 2$ **do**
  pop index i; let $p \leftarrow prev(i)$, $q \leftarrow next(i)$
  if p and s_i share y **and** x_i lies strictly between x_p and x_q **then**
    remove s_i; enqueue p and q
  else if s_i and q share x **and** y_i lies strictly between y_p and y_q **then**
    remove s_i; enqueue p and q
  end if
end while
if IsCanonical(S) is false **then**
  return failure
end if
return S

4 removes only such points, returns a canonical list when it succeeds. Its running time is $O(m)$.

Proof. A removed point lies in the interior of a straight boundary edge, so deleting it merges two collinear edges into a single edge; the polygon is unchanged as a point set, so its perimeter and area are unchanged. The betweenness test is exactly the condition that the point is interior to that edge. Without the test, a spike such as $((0,0),(2,0),(1,1))$ would lose the segment from $(1,0)$ to $(2,0)$; the spike has the same area but a different perimeter and boundary. The first pass removes only cyclic consecutive duplicates, which are zero-length edges and have the same preservation property. Each point is removed at most once and only the two neighbours of a removed point are re-examined, so the work queue performs $O(m)$ tests in total; the final canonicality check is a single $O(m)$ pass. The final check is a safety net for an unexpected non-canonical configuration. □

The restart-based version of the betweenness scan is $O(m^2)$ worst case, because it restarts an $O(m)$ scan after each removal. The work-queue implementation above is $O(m)$ and is the canonicalizer exposed by the implementation. Canonicalization is not a simplicity certificate: it removes only points whose deletion preserves the boundary. The C++ implementation throws only if the final safety check fails.

D. Composition and Applications

The hull pass has two uses. The first is independent of creation: it computes the x- or y-monotone hull of a stored polygon or point set, and composing the two passes yields the rectilinear convex hull. The same construction gives a monotone enclosure or bounding shape and is a useful preprocessing step for decomposition. The second use is to compute a monotone or convex hull explicitly.

Applying the x-monotone hull and then the y-monotone hull produces a representation that is both x- and y-monotone on the tested fixtures; the composition is checked for monotonicity and canonicality before it is used as a decomposition input. Figure Fig. 7 shows the resulting rectilinear convex hull of a point set.

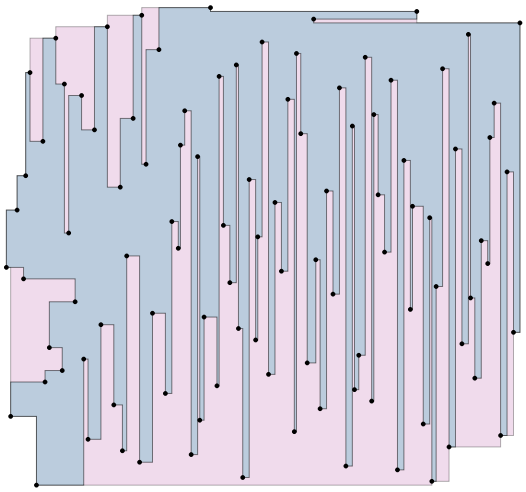


Figure 7: The rectilinear convex hull of a point set, obtained by the x- then y-monotone hull passes.

VI. Recursive Cut Framework

A. Data Structure

All cut policies operate on a circular doubly linked list whose nodes are the stored points. A cut inserts a new node into the list and appends the corresponding point to a point array. The recursive procedure returns lists of node indices; the wrapper converts each index list into a polygon. The linked list supports insertion and traversal in constant and linear time, respectively, and avoids the index shifting that an array-based implementation would require.

B. Concave Predicate

For three consecutive stored points p_0, p_1, p_2, the explicit and convex policies use the direction-reversal guard

$$R(p_1) = \big[(p_1^x - p_0^x)(p_2^x - p_1^x) < 0\big] \vee \big[(p_1^y - p_0^y)(p_2^y - p_1^y) < 0\big] \tag{10}$$

and the signed-area term

$$a(p_1) = (p_1^y - p_0^y)(p_2^x - p_1^x). \tag{11}$$

Algorithm 5 FindConcave(v, π)

Require: root v, policy π
$c \leftarrow v$
while True **do**
  $p_0 \leftarrow prev(c)$; $p_1 \leftarrow c$; $p_2 \leftarrow next(c)$
  if Accept$_\pi(p_0, p_1, p_2)$ **then**
    return c
  end if
  $c \leftarrow next(c)$
  if $c = v$ **then**
    return NIL
  end if
end while

Algorithm 6 FindMinDist(v)

Require: concave vertex v
$best \leftarrow \infty$; $target \leftarrow$ NIL; $vertical \leftarrow$ True
for each stored node q **do**
  $q^- \leftarrow prev(q)$; $q^+ \leftarrow next(q)$
  if $q_y^- < v_y \le q_y$ **or** $q_y \le v_y < q_y^-$ **then**
    $d \leftarrow |q_x - v_x|$
    if $d < best$ **then**
      $best \leftarrow d$; $target \leftarrow q$; $vertical \leftarrow$ True
    end if
  end if
  if $q_x^+ < v_x \le q_x$ **or** $q_x \le v_x < q_x^+$ **then**
    $d \leftarrow |q_y - v_y|$
    if $d < best$ **then**
      $best \leftarrow d$; $target \leftarrow q$; $vertical \leftarrow$ False
    end if
  end if
end for
return $(target, vertical)$

The vertex is treated as concave when $R(p_1)$ holds and

$$\mathrm{cmp}(a) = \begin{cases} a > 0, & \text{if the polygon is counter-clockwise,} \\ a < 0, & \text{if the polygon is clockwise.} \end{cases} \tag{12}$$

This is the compressed-list form of the reflex test. It should not be read as a test for every reflex vertex of the reconstructed boundary. The inner corner of an L-shape is a reflex vertex of the full boundary, yet the compressed predicate does not cut it, because the L-shape is already orthogonally convex. The predicate detects a stored corner at which the compressed chain fails the monotonicity condition; its role is to drive the partition into orthogonally convex pieces, not to enumerate all 270° corners.

C. Helper Operations

The skeleton uses two helper operations. We write v_x, v_y for the coordinates of a point v, and q^-, q^+ for the predecessor and successor of a stored point q. FindConcave scans stored triples in cyclic order and returns the first vertex accepted by the policy. FindMinDist scans stored points and their incident edges, projects the concave vertex onto each edge that spans its coordinate, and returns the nearest projection together with its direction.

Table VI defines the three policy predicates and their contracts. The convex and explicit policies use the stored-triple guard (10) and area term (11); the implicit policy uses the stored-pair sign test (17). The explicit policy is defined for a general simple polygon; the implicit policy is

Algorithm 7 CUT(v)

Require: a circular list of stored points rooted at v
Ensure: a list of pieces
if v has exactly two nodes **then**
  return the two-node piece
end if
if the policy allows a three-node base case and v has three nodes **then**
  return the three-node piece
end if
$v_{\text{cur}} \leftarrow$ FINDCONCAVE(v)
if $v_{\text{cur}} =$ NIL **then**
  return the whole cycle as one piece
end if
$(v_{\text{min}}, \textit{vertical}) \leftarrow$ FINDMINDIST(v_{cur})
if a degenerate two-node split applies **then**
  return SPLIT($v_{\text{cur}}, v_{\text{min}}$)
end if
$p_{\text{new}} \leftarrow$ INSERTCUT($v_{\text{cur}}, v_{\text{min}}, \textit{vertical}$)
$u \leftarrow$ FIRSTTARGET($v_{\text{cur}}, v_{\text{min}}$)
return CUT(u) $\cup$ CUT(p_{new})

documented for a convex input, which is the contract used in Section VIII.

D. Skeleton

Algorithm 7 shows the shared skeleton. It is a template method: the policies differ in the base cases, in the concave-vertex predicate, and in the target of the first recursive call. The convex policy is described in Section VII; the explicit and implicit policies reuse the skeleton with different predicates in Section VIII.

The base cases are stored pieces, not expanded boundaries. For a simple input polygon, a two-node piece is a non-degenerate rectangle (its two points differ in both coordinates); a three-node piece is an L-shape when the expansion is simple. Canonicality alone also admits a distinct two-point list that shares an axis; it is a degenerate segment, not a simple polygon, and is not a decomposition base case. The predicate `FindConcave` returns the first stored vertex that violates the policy's concavity test. If no such vertex exists, the current cycle is returned as a single piece. The nearest-edge search returns a node v_{min} and a flag indicating whether the cut is vertical. The cut is inserted as a new node; the two recursive calls then process the two sides. The `FirstTarget` hook is the only difference between the explicit and implicit recursions: the explicit policy recurses first on v_{cur}, the implicit policy on v_{min}.

E. Nearest Edge and Cut Insertion

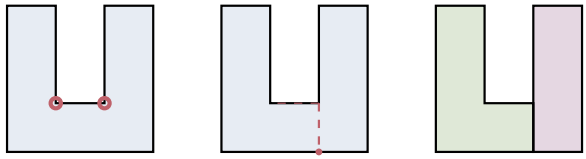
Given a concave vertex v, the algorithm projects v vertically and horizontally onto the boundary and selects the nearest edge. For the explicit and convex policies the projection point is v itself; for the implicit policy it is the implicit corner $c = (x_{\text{next}}, y_{\text{cur}})$. The search examines each edge in the circular list and keeps the minimum distance. If the nearest edge is vertical, the new cut point is

$$p_{\text{new}} = (x_{\text{min}}, y_{\text{cur}}); \tag{13}$$

if it is horizontal,

$$p_{\text{new}} = (x_{\text{cur}}, y_{\text{min}}). \tag{14}$$

The new node is spliced between the appropriate neighbours. The update replaces the target edge by two edges through p_{new} and links p_{new} to the concave vertex, so the circular list splits into two cycles that share p_{new} and v; the four affected links are updated in $O(1)$. The projection condition in the search is the discrete visibility test: it selects an edge that spans the coordinate of v, so the segment from v to the projection point lies inside the polygon. The implementation uses a strict inequality to break ties and to avoid selecting an edge incident to v itself. Lemma 4 proves that the projection is the first boundary hit and that the open segment lies inside the polygon.



(a) concave vertices (b) minimum cut (c) split and recurse

Figure 8: The cut primitive: (a) concave vertices; (b) nearest visible edge; (c) split and recurse.

F. Degenerate Cuts

A cut point can coincide with an existing vertex. When the nearest target is adjacent to the current vertex in the circular list and the cut point duplicates that target, inserting a new node would create a zero-length edge. The implementation detects this situation and splits off the existing two-node piece instead of inserting a duplicate. The twelve-point fixture exercises this case: the first cut at the stored vertex $(3, 2)$ targets $(1, 2)$, which is adjacent in the list, so the two-node piece $(2, 3), (1, 2)$ is split off as a unit. The remaining two pieces are the area-10 piece (six stored points in the raw output, five after canonicalization) and the five-point piece of area 9. Without the degenerate split, the same run returns a single nineteen-area piece and a one-area piece, and the three-piece partition is lost.

The convex wrapper can also receive a zero-width two-node sliver on a mixed fixture. Such a sliver is a distinct two-point list that shares an axis; it is canonical but not a simple polygon. It has zero area and is discarded by the wrapper; it is not a piece of the output. Because the sliver has empty interior, discarding it does not change the partition of the polygon's interior or the area sum. The test suite records this behavior.

VII. Convex Decomposition

A. Algorithm

The convex policy cuts until no stored triple satisfies the predicate of Section VI. Its base cases are the two-node rectangle and the three-node monotone piece. The recursion is the skeleton of Algorithm 7 with the convex predicate and `has_triangle_base_case = true`.

Table VI: Cut policies. The convex and explicit predicates use $R(p_1)$ and $a(p_1)$ from (10)–(11); the implicit predicate uses (17).

Policy	Predicate	Base case	First target	Input contract
Convex	$R(p_1) \wedge \mathrm{cmp}(a(p_1))$	2 or 3	v_{cur}	simple polygon
Explicit	$R(p_1) \wedge \mathrm{cmp}(a(p_1))$	2	v_{cur}	simple polygon
Implicit	$\mathrm{cmp}(a(c_i))$	2	v_{min}	convex input

B. Correctness

Lemma 4 (cut partition). A cut from a concave vertex to the first boundary point hit by an interior axis-parallel ray splits a simple rectilinear polygon into two simple rectilinear polygons whose interiors are disjoint and whose areas sum to the area of the original polygon.

Proof. Let v be the concave vertex and let p be the first boundary point hit by the ray from v into the interior. Such a point exists because the polygon is bounded. The open segment (v, p) cannot meet the boundary: if it did, the first such intersection would be closer to v than p; if it left the interior, it would cross the boundary at a closer point. Hence (v, p) lies in the interior. The nearest-edge scan of Algorithm 6 considers exactly the edges that cross the ray. For a vertical target edge the distance along the ray is $|q_x - v_x|$, and for a horizontal target edge it is $|q_y - v_y|$; the minimum over the scanned edges is therefore the first hit p. The strict inequalities in the scan exclude the two edges incident to v. Adding the segment vp to the boundary produces two closed cycles. Each cycle is simple because vp meets the boundary only at its endpoints and does not cross the boundary in its interior. The interiors of the two cycles are disjoint and their union is the interior of the original polygon. The signed-area sum is additive because the segment vp is traversed in opposite directions in the two cycles. □

Lemma 5 (convexity criterion). Let Q be a simple orthogonal polygon with stored sequence S. If Q is not orthogonally convex, then some stored triple satisfies the convex predicate of Section VI.

Proof. If Q is not orthogonally convex, an axis-parallel line L meets Q in at least two disjoint intervals. Take two consecutive intervals and let G be the gap between them. Walk along the boundary of Q from one side of G to the other; the first reflex vertex v encountered is the innermost corner of a dent that separates the two intervals. If v is stored, let p_0 and p_2 be its stored predecessor and successor. The two incident edges at v turn in opposite coordinate senses, so $R(v)$ holds; because v is reflex, the signed area term $a(v)$ has the sign selected by cmp, so the predicate accepts v. If v is implicit, write $v = I_i$ for the stored pair s_i, s_{i+1}. The four possible orientations of the two incident edges are checked directly: at the stored endpoint s_i (or s_{i+1}) the guard R and the sign of a are exactly the sign of the omitted reflex corner, so a stored triple is accepted. Thus a non-orthogonally-convex piece always exposes an accepted stored triple. □

Theorem 1 (convex decomposition). The convex recursion terminates and returns a partition of the input polygon into orthogonally convex pieces. The area of the pieces sums to the area of the input.

Proof. Each cut is a non-crossing chord of the current simple polygon (Lemma 4). A simple polygon with n vertices admits at most $n - 3$ non-crossing chords, so the recursion performs $O(n)$ cuts and terminates; the base cases stop on two or three stored points. Lemma 4 gives the partition and the area identity. At termination no stored triple satisfies the convex predicate, so Lemma 5 implies that the piece is orthogonally convex. A zero-area sliver has empty interior and is discarded without changing the partition of the interior. □

C. Example

The twelve-point fixture has stored points

$$(-1,1),(0,0),(1,-1),(3,0),(4,1),(6,2),(5,5),(4,4),(3,2),(2,3),(1,2) \tag{15}$$

and area 20. The convex policy returns three pieces, listed in Table VII. The two-node piece is a rectangle; the other two are orthogonally convex but not rectangles. These pieces are the input to the rectangle stage of Section IX.

The C++ convex policy returns the six-point list shown for the area-10 piece. The canonicalization pass removes the redundant collinear point $(3, 0)$ from the raw six-point list, yielding the five-point list; the perimeter, the area, and the output class are unchanged. This is a concrete example of why canonicalization is not a simplicity certificate.

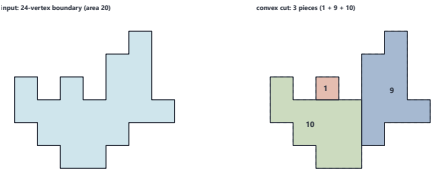


Figure 9: Convex decomposition of the twelve-point fixture: three orthogonally convex pieces of areas 1, 9, and 10.

VIII. Explicit and Implicit Policies

A. Explicit Policy

The explicit policy uses the same recursive skeleton and the same concavity predicate as the convex policy, but it omits the three-node base case. It handles the stored 270° vertices: it continues until no stored triple satisfies the predicate. In the three-step rectangle algorithm, the explicit phase is applied before the implicit phase, so the stored corners are resolved first and the implicit phase only has to handle the corners introduced by the expansion.

Table VII: Convex decomposition of the twelve-point fixture. The areas sum to 20.

Piece	Stored points	Area
Rectangle	$(2,3),(1,2)$	1
Orthogonally convex	$(3,2),(0,3),(-1,1),(0,0),(1,-1),(3,0)$	10
Orthogonally convex	$(3,0),(4,1),(6,2),(5,5),(4,4)$	9

B. Implicit Policy

The implicit policy operates directly on stored pairs. For a stored point $p_1 = S_i$ and its successor $p_2 = S_{i+1}$, the corner under test is the implicit vertex

$$c_i = (x_{i+1}, y_i). \tag{16}$$

The C++ API documents this entry point as a cut of a *convex* polygon; the policy assumes a convex input, because it tests only implicit corners. It is applied to the convex pieces produced by the convex stage.

The policy uses the sign test

$$a(c_i) = -(y_{i+1} - y_i)(x_{i+1} - x_i) \tag{17}$$

and the orientation comparison of (12). The implicit vertex is a genuine 90° corner by construction, so the sign test alone decides whether it is reflex in the orientation of the polygon.

For example, the L-shape stored as $((0,0),(2,1),(1,2))$ has one implicit corner, $(1,1)$. The relevant stored pair is $((2,1),(1,2))$. The test value is

$$a = -(2-1)(1-2) = +1, \tag{18}$$

which is positive, so the counter-clockwise orientation selects the implicit corner. A horizontal cut yields the rectangles $((0,0),(2,1))$ and $((0,1),(1,2))$ of areas 2 and 1.

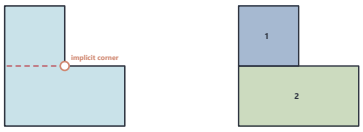


Figure 10: Implicit cut of an L-shape. The implicit corner $(1,1)$ is reflex; a horizontal cut yields two rectangles of areas 2 and 1.

C. Explicit and Implicit Corners

The explicit policy handles stored 270° vertices; the implicit policy handles omitted 270° vertices. The two cases are disjoint: a stored point is not an omitted point, and an omitted point is not stored. In the three-step rectangle algorithm of Section IX, the explicit phase is applied first, so the stored corners are resolved before the implicit phase; the implicit phase then handles the corners introduced by the expansion. The L-shape example above shows the implicit phase; a stored reflex corner is handled by the explicit phase in the same way. Together the two phases cover both types of 270° vertex.

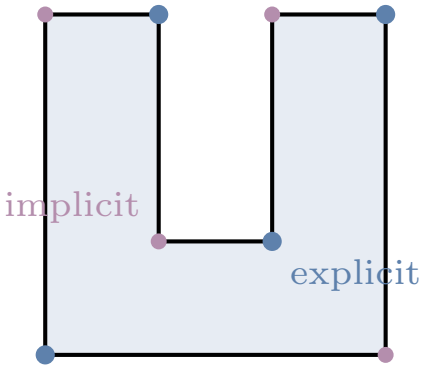


Figure 11: A U-shape contains both explicit 270° stored vertices (blue) and implicit 270° vertices created by the expansion (purple).

IX. Rectangle Decomposition

A. Three Steps

A rectangle decomposition is specified by three steps.

1. **Convex decomposition.** Compute a partition of the input into orthogonally convex pieces. This step makes the next two steps local: each piece is x- and y-monotone, and every remaining 270° vertex lies on one of the two monotone chains. A cut at such a vertex therefore splits the piece into two x-monotone pieces.
2. **Handle explicit 270° vertices.** A vertex is *explicit* if it is a stored point and its interior angle is 270°. For each explicit 270° vertex, cut from the vertex to the nearest visible edge (or vertex), split the piece, and recurse. Repeat until no explicit 270° vertex remains.
3. **Handle implicit 270° vertices.** A vertex is *implicit* if it is an omitted vertex $I_i = (x_{i+1}, y_i)$ and its interior angle is 270°. Cut it in the same way and recurse. Repeat until no implicit 270° vertex remains.

A simple orthogonal polygon with no 270° vertex is a rectangle. Steps 2 and 3 are repeated until neither type remains; a cut can create a new 270° vertex of the other type, and the corresponding step handles it. The three steps therefore specify a rectangle decomposition.

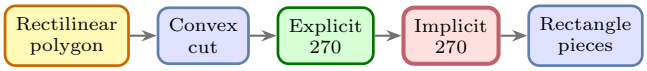


Figure 12: The three-step rectangle decomposition pipeline.

B. Correctness

Theorem 2 (rectangle decomposition). Let P be a simple rectilinear polygon. Apply the convex decomposition of Section VII; then apply the explicit phase to every stored 270° vertex and the implicit phase to every 270° vertex introduced by the expansion, repeating the two phases until neither type remains. The output is a partition of P into rectangles, and the areas of the rectangles sum to $A(P)$.

Proof. Theorem 1 partitions P into simple orthogonal pieces. In any such piece, every 270° vertex is a boundary vertex; it is either a stored point or an omitted point I_i. Step 2 cuts the stored 270° vertices and step 3 cuts the omitted ones; repeating the two phases handles vertices created by a cut. When neither type remains, every interior angle is 90°. A simple orthogonal polygon has $n = 2r + 4$ vertices, where r is the number of 270° vertices; with $r = 0$ we have $n = 4$, and a simple orthogonal quadrilateral is a rectangle. Every cut is a chord inside the current piece (Lemma 4), so the pieces partition P and their areas add. □

The seven-rectangle result in Table VIII is the output of this pipeline on the twelve-point fixture.

C. Example

Applying the pipeline to the twelve-point fixture yields seven rectangles, listed in Table VIII. Every rectangle has two stored points with distinct x- and y-coordinates, and the areas sum to 20. This is the non-degenerate two-point case; canonicality alone also admits a two-point shared-axis list, which is not a rectangle.

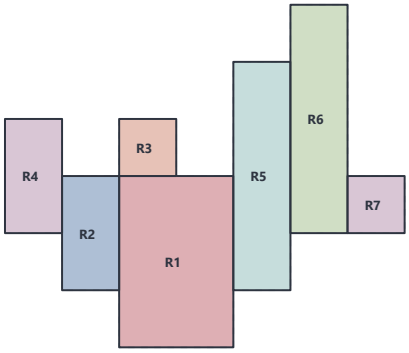


Figure 13: Rectangle decomposition of the twelve-point fixture: seven rectangles of areas 1, 1, 2, 2, 4, 4, and 6.

D. Remarks

The algorithm is not a minimum-cardinality rectangle partition. A carefully chosen cut sequence can use fewer rectangles. The goal here is a simple, deterministic, and provably correct pipeline that is easy to verify against the output class. Minimum rectangle partition is a separate optimization problem [9], [10], [12].

The implementation provides the convex, explicit, and implicit phases and the three-step rectangle pipeline. The seven-rectangle result in Table VIII and Figure Fig. 13 is produced by this pipeline.

X. Complexity

Let n be the number of boundary vertices and $m = n/2$ the number of stored points. We state the bounds in m.

Canonicality. The canonicality check is a single $O(m)$ pass: it scans the cyclic consecutive stored points for duplicates first, then for betweenness-redundant shared-axis pairs. No sorting or all-pairs scan is needed.

Creation and canonicalization. The creation sort is $O(m \log m)$. Canonicalization of a creation ordering is $O(m)$ because each point is removed at most once and only its neighbours are revisited.

Monotone hull. Each sweep is $O(m)$; the hull's restart-based collinear scan is $O(m^2)$ worst case.

Cut stage. The nearest-edge search scans $O(m)$ stored edges. Each cut increases the number of pieces by one and the recursion creates at most $O(m)$ pieces, so the cut stage takes $O(m^2)$ time. The linked list supports insertion in $O(1)$ and the point array is appended in amortized $O(1)$. The recursion uses $O(m)$ stack space in the worst case and $O(m)$ storage for the pieces.

Rectangle stage. Each explicit or implicit cut scans $O(m)$ stored points and edges, and the recursion creates at most $O(m)$ pieces, so the rectangle stage takes $O(m^2)$ time in the worst case. The output has $O(m)$ rectangles because every cut increases the number of pieces by one.

Overall. The pipeline runs in $O(m^2)$ time and $O(m)$ extra space. The asymptotic order is the same as on the full vertex list, with $n = 2m$ substituted; Section IV-B gives the per-pass constants, and Section IV-C gives the storage. Canonicalization is linear and is not the source of the quadratic term. On the twelve-point fixture the convex stage returns three pieces after two cuts, and the rectangle stage returns seven rectangles.

XI. Implementation and Verification

A. Implementations

The algorithms are implemented in C++20 in `physdes-cpp` (the `Recti` library). The implementation provides `is_canonical` (a single $O(m)$ pass), `canonicalize` (an $O(m)$ work-queue pass that removes consecutive duplicates and redundant collinear points), `rpolygon_expand` (the x-first expansion to the full boundary), the creation and canonicalization passes, the convex, explicit, and implicit phases, the three-step rectangle pipeline, and the full-vertex baseline `rpolygon_cut_convex_full`. It uses the twelve-point fixture and the area/orientation conventions described above.

The main entry points are `is_canonical` and `canonicalize`; the creation routines

Table VIII: Rectangle decomposition of the twelve-point fixture. The areas sum to 20.

Rectangle	Opposite corners	Area
R_1	$(1,-1),(3,2)$	6
R_2	$(0,0),(1,2)$	2
R_3	$(1,2),(2,3)$	1
R_4	$(-1,1),(0,3)$	2
R_5	$(3,0),(4,4)$	4
R_6	$(4,1),(5,5)$	4
R_7	$(5,1),(6,2)$	1

`create_xmono_rpolygon` and `create_ymono_rpolygon`; the hull routines `rpolygon_make_xmonotone_hull`, `rpolygon_make_ymonotone_hull`, and `rpolygon_make_convex_hull`; the cut routines `rpolygon_cut_convex`, `rpolygon_cut_explicit`, `rpolygon_cut_implicit`, and `rpolygon_cut_rectangle`; and the full-vertex baseline `rpolygon_cut_convex_full`. The creation routines return an orientation flag; the x-monotone variant reports anti-clockwise and the y-monotone variant reports clockwise, so the tests check the sign explicitly.

The tests build and run with `xmake build test_recti` and `xmake run test_recti`. The suite is the `test_recti` target in the current working tree of the `dev` branch of `physdes-cpp`, plus the test fixtures described in this paper.

B. Test Design

The tests cover seven layers.

1. **Canonicality tests.** The canonical-form cases, including $m < 2$, the distinct two-point shared-axis case, the two-point duplicate, $m \geq 3$ betweenness-collinear points, the shared-axis cases, and the non-adjacent duplicate case.
2. **Canonicalization tests.** `canonicalize` removes consecutive duplicates, removes a betweenness-collinear point, preserves the signed area (and, by Lemma 3, the perimeter), and keeps a non-redundant point.
3. **Creation and canonicalization tests.** Create an ordering, assert its monotonicity, apply canonicalize, and assert canonicality of the canonicalized list. The twelve-point orderings are included.
4. **Hull tests.** Assert monotonicity and canonicality of the hull outputs.
5. **Cut tests.** Assert orthogonal convexity on the *expanded* pieces and check signed-area preservation; assert canonicality on the pieces where the implementation guarantees it (the random, AGPLIB, and bridged-hole fixtures). The twelve-point fixture is required to return three pieces of areas 1, 9, and 10; its raw pieces may contain implicit collinear points, so canonicality is not asserted for them, only convexity and area. The mixed fixture that produces a zero-width sliver is required to return two valid pieces; the sliver is canonical but not simple and is discarded. A bridged-hole fixture $(4,1),(10,7),(7,5),(9,2),(6,5),(7,7)$ is cut into three pieces of areas $2, 15, 10$; the bridge is traversed twice and the area is preserved. Two AGPLIB instances, `rand-10-1` and `rand-12-6` [20], are cut into two and three orthogonally convex pieces with area preservation. The C++ suite includes the two AGPLIB fixtures and the bridged-hole fixture.
6. **Rectangle tests.** Assert that every output piece is a two-point rectangle with distinct coordinates and positive area; the twelve-point fixture must return seven rectangles of areas 1, 1, 2, 2, 4, 4, and 6, and the area sum must be 20.
7. **Full-vertex cut tests.** Expand the stored sequence with `rpolygon_expand`, run `rpolygon_cut_convex_full`, and assert that every piece is orthogonally convex and that the areas sum to the input area; the twelve-point fixture, a family of comb fixtures, and a 200-point random polygon are included.

The convexity oracle is deliberately not `rpolygon_is_convex(piece)` on the stored points. That call checks the stored points, not the implicit vertices, and it returns true for any stored list of length at most three. The tests expand the piece first and check orthogonal convexity on the full boundary. This is the oracle that catches a zero-length edge in the six-point cut piece. A canonical two-point shared-axis list expands to a degenerate boundary; the cut wrapper discards it as a zero-width sliver rather than treating it as a piece.

C. Representation Micro-benchmark

The comparison in Section IV-B is measured by `bench/bench_repr.cpp` in `physdes-cpp`, built and run with

```
xmake build bench_repr
xmake run bench_repr
```

The fixture is a reduced, simple, orthogonally convex staircase with $n = 2k + 2$ vertices and $m = k + 1$ stored points; the stored sequence is `full[::2]`, and the benchmark asserts `rp.signed_area() * 2 == pg.signed_area_x2()` before timing. Both classes therefore carry the same polygon, use the same 8-byte point layout, and differ only in the number of stored units. The two calls are the repository's current telescoping implementation; both use the same formula and differ only in input length. Each measurement is the median of seven trials with adaptive repetitions (at least 20 ms per trial) on an Intel Core i9-13900H running Windows x64, compiled by MSVC 19.51

in release mode. Table IX reports one representative run; the byte counts are in Table V. We report $m \geq 128$: below that, the fixed per-call cost dominates and the ratio is not informative.

Three conclusions follow. First, the byte ratio reaches 2.00 and the bounding box reaches 2.00: these are the passes whose per-unit work is identical, and they confirm the counting model. Second, area lands between 1.1 and 1.7 across sizes (about 1.3 at $m = 8192$), and canonicality at about 1.3: the outer-loop count halves, but the per-iteration constants do not, and canonicality compares two different predicates. Third, expansion at $m = 8192$ costs 138–183 μs across runs, while the full-vertex area pass costs 14–19 μs; a workload that expands on every operation would lose more than it gains. These measurements are consistent with the thesis of Section IV: the representation is a memory and loop-count optimization, not a universal constant-factor speedup. Table IX is one representative run; the ranges above are the run-to-run variation.

D. Convex Decomposition on the Full Vertex List

To compare the decomposition itself, we implement a full-vertex baseline, `rpolygon_cut_convex_full`, in `source/rpolygon_cut_full.cpp`. It stores every boundary vertex, tests orthogonal convexity with an $O(k)$ x- and y-monotonicity scan, and, when a piece is not convex, cuts at the first reflex vertex along an interior axis to the nearest edge, splits the cycle into two, and recurses. Each returned piece is canonicalized. The even-index input is expanded with `rpolygon_expand` before it is passed to the baseline, so both algorithms see the same polygon. The comparison builds and runs with

```
xmake build bench_cut
xmake run bench_cut
```

Table X reports one run; every piece in every row passed the area-sum and orthogonal-convexity checks. The comb fixtures use the same number of pieces in both versions; the twelve-point fixture differs because the full-vertex version cuts at the first reflex vertex (five pieces) while the even-index version cuts at a stored vertex (three pieces). The full-vertex version is 2–12× slower. Every row runs both versions on the *same* polygon: the range is across fixture sizes and shapes, not across different polygons at the same size. The ratio grows with the fixture (from 2.0× at $n = 12$ to 11.9× at $n = 260$) because the full-vertex version tests orthogonal convexity globally, with two monotonicity scans per recursion, while the even-index version uses a local predicate; both versions use the same linked list and $O(1)$ split.

Part of the gap is algorithmic and part is representational. The full-vertex version stores $n = 2m$ points, so every scan covers twice as many units; it also tests convexity globally (two monotonicity scans over the current cycle) because an orthogonal polygon can have a reflex vertex and still be orthogonally convex (an L-shape), so a single local reflex test cannot decide the base case. The even-index version's three-point predicate is local by construction: the stored triple spans a two-edge macro-step. Both versions now use the same linked list and $O(1)$ split. The 2–12× ratio therefore reflects the global convexity test and the 2× point count, not a data-structure artifact. The full-vertex baseline returns valid orthogonally convex pieces and is the natural implementation when the input is already a full vertex list. input is already a full vertex list.

E. Results

Table XI reports the suite sizes and the fixture results. The C++ suite passes 147 cases with 1431 assertions under `xmake`.

The twelve-point fixture is decomposed into seven rectangles of areas $1, 1, 2, 2, 4, 4, 6$, all summing to 20. The tests assert output-class properties (canonicality where guaranteed, orthogonal convexity, and rectangle shape), not only the area; this is what distinguishes a correct rectangle decomposition from a collection of convex pieces with the right total area. The three raw even-index fixture pieces are the exception: their convexity and area are asserted, but canonicality is not, because they may contain implicit collinear points.

F. Threats to Validity

The evaluation uses generated point sets and hand-built fixtures, not foundry data. The claims are algorithmic: exactness, termination, canonicality, perimeter and area preservation, and the output class. They are not signoff claims. Canonicality is local and does not imply simplicity; simplicity is an assumption on the input and is checked separately where the implementation needs a guard. The implementation is not cross-validated against an independent oracle on random inputs; the shared fixture and the same area/orientation conventions are the current check. It uses `int` coordinates; the intended range is small, and overflow is not handled. The representation benchmark uses one machine, one compiler, and a single family of staircase fixtures; its absolute times and its area ratio are indicative rather than portable constants. The byte ratio and the bounding-box ratio follow from the record count and the x-first map, so they are the stable part of the measurement.

XII. Discussion and Limitations

Representation. The encoding is a reparameterization, not an information-theoretic compression: the omitted vertices carry no independent information, and the stored count halves because a reduced boundary has n degrees of freedom, not $2n$. Canonicality is a local property of the encoding, not a simplicity certificate: a canonical list can repeat a non-adjacent point or expand to a self-intersecting boundary. Simplicity is an assumption on the input and is checked separately where the implementation needs a guard. The encoding depends on the starting

Table IX: Measured time in nanoseconds (median of seven trials). "Ratio" is full/even; values above one favour the even-index encoding. One representative run; ranges across five runs are given in the text.

m	n	area even	area full	ratio	bbox even	bbox full	ratio	canon even	canon full	ratio
128	256	212.5	300.0	1.41	262.5	506.3	1.93	2837.5	3675.0	1.30
512	1024	687.5	1181.3	1.72	1000.0	1950.0	1.95	11100.0	14700.0	1.32
2048	4096	2675.0	3581.3	1.34	3875.0	7676.6	1.98	44325.0	59656.3	1.35
8192	16384	10625.0	14100.0	1.33	15412.5	30806.3	2.00	177656.3	238240.6	1.34

Table X: Convex decomposition: even-index versus full-vertex. Both versions preserve the area and return orthogonally convex pieces. Times are medians; one run.

Fixture	m	n	Even pieces	Full pieces	Even cut (μs)	Full cut (μs)	Ratio
Twelve-point	12	24	3	5	4.4	10.8	2.5×
Comb, $k = 2$	6	12	3	3	2.6	5.1	2.0×
Comb, $k = 4$	10	20	5	5	4.6	9.9	2.2×
Comb, $k = 8$	18	36	9	9	8.3	25.3	3.1×
Comb, $k = 16$	34	68	17	17	15.5	68.9	4.5×
Comb, $k = 32$	66	132	33	33	29.7	213.6	7.2×
Comb, $k = 64$	130	260	65	65	66.3	788.3	11.9×

Table XI: Verification results.

Suite / check	Cases / assertions	Result
C++ `test_recti`	147 cases, 1431 assertions	pass
Canonicality	Table I	pass
Canonicalization	duplicates, betweenness	pass
Area preservation	all canonicalization tests	pass
Convex cut, twelve-point fixture	3 pieces; areas 1, 9, 10	pass
Full-vertex convex cut	twelve-point, combs, random 200-point	pass
Rectangle cut, twelve-point fixture	7 rectangles; areas 1, 1, 2, 2, 4, 4, 6	pass

vertex and on the orientation; the algorithms use the given rotation and an orientation flag. `is_canonical` is the canonicality predicate; `canonicalize` is an $O(m)$ canonicalization pass that removes consecutive duplicates and redundant collinear points.

Polygons with holes. A polygon with holes is reduced to a single boundary by a bridge cut from the outer boundary to each hole. The bridge is traversed in both directions, so the area is unchanged; the boundary is weakly simple for a zero-width bridge and simply connected for a narrow bridge. The even-index encoding and the cut skeleton then apply to that boundary, and repeated bridge vertices are handled by the degenerate split; non-adjacent repeats are canonical and do not affect the canonicality check. This is standard and is not specific to the even-index representation. The proofs are stated for a simply connected input; the bridge reduction supplies that case.

When the full-vertex list wins. The full list is the better choice in four situations. (i) *Dynamic geometry.* Inserting or deleting an arbitrary boundary vertex, or splitting an edge at a T-junction, changes the parity of the alternating class and forces a re-encoding; a full list accepts the vertex in $O(1)$. (ii) *Non-rectilinear geometry.* A 45° edge breaks the alternation on which the x-first rule depends. (iii) *Weakly simple boundaries.* A zero-width bridge creates a spike and repeated vertices, so reducedness fails; the even-index form must either keep the spike, losing canonicality, or use a non-reduced variant. Separate loops or a full walk are simpler. (iv) *Interchange and full-boundary workloads.* GDSII and OASIS consume vertex lists, and every expansion costs $O(m)$, measured at 7–13× the full-vertex area pass at $m = 8192$. If each operation expands, the representation is a net loss. The encoding is therefore a representation for the *interior* of an algorithm, not for its interface.

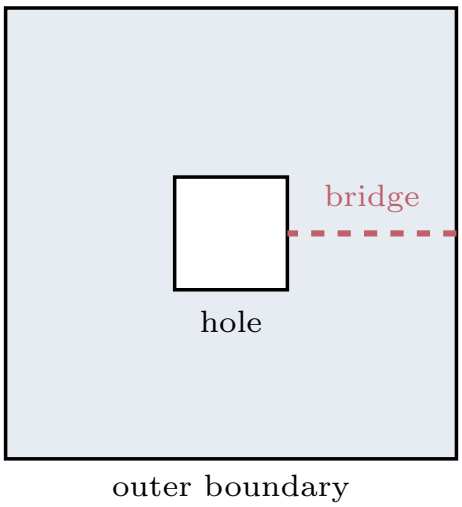


Figure 14: A bridge cut from the outer boundary to a hole merges the two boundaries into one cycle; the bridge is traversed in both directions.

No optimality claim. The rectangle pipeline is a cut-based decomposition, not a minimum-cardinality partition. The seven rectangles on the fixture are not claimed to be minimal. Minimum rectangle partition is a different problem with a rich literature [9], [10], [12]; the goal here is a simple, verified preprocessing pipeline.

Proof scope. The proofs assume a simply connected input and a reduced boundary. They cover canonicality (local, not a simplicity certificate), perimeter and area preservation under canonicalization, $O(m)$ canonicalization, termination, partition, the convexity criterion, orthogonal convexity of the convex stage, and rectangle output. A polygon with holes is reduced to this case by the bridge cut described above; minimum-cardinality optimization is not addressed.

Quadratic scaling. The decomposition is $O(m^2)$, where m is the number of stored points. For $m \leq 100$ the quadratic algorithm is appropriate and often faster because its constant is small; for $m > 10^3$ a sub-quadratic method would be preferable. The literature on minimum/near-minimum rectangle partition is relevant, but the present pipeline is not optimized for that regime. Canonicalization itself is linear and is not the source of the quadratic term.

Degeneracies. Zero-width slivers, duplicate implicit points, and collinear stored points are handled by canonicalization (consecutive duplicates first, then betweenness-collinear points), the degenerate two-node split, and the zero-width discard. A distinct two-point shared-axis list is canonical but not simple; the cut wrapper discards it as a zero-width sliver. The treatment is not exhaustive: cuts through vertices and cuts along the boundary still need separate case analysis. Exact integer arithmetic avoids rounding but not these combinatorial degeneracies; symbolic perturbation [19] is a complementary technique.

XIII. Conclusion

We asked what the x-first even-index encoding actually buys relative to the full vertex list. The answer has three parts. It is *reversible*: the x-first rule recovers the full boundary in $O(m)$, so the halving costs no information. It *halves storage and loop counts*: with the same 8-byte point layout the measured byte ratio reaches 2.00, the bounding box is 1.93–2.00× faster, and every $O(n)$ pass becomes $O(m)$. It is *not a universal speedup*: area measured about 1.3× at the largest sizes and canonicality about 1.3×, and expanding the full boundary costs 7–13× the full-vertex area pass at $m = 8192$, so a workload that expands on every operation is better served by the full list. The encoding also makes the decomposition base cases explicit: two stored points are a rectangle, three are an L-shape, and the three-step pipeline returns a partition into rectangles in $O(m^2)$ while passing 147 cases with 1431 assertions. The full-vertex convex baseline returns valid orthogonally convex pieces on the same fixtures but is 2–12× slower, with the gap growing with size: it stores twice as many points and must test convexity globally, whereas the even-index version uses a local predicate. Source size does not favour the encoding either: for canonical output the even-index path is 268 NBNC against 220 for the full-vertex module. The practical lesson is to use the encoding inside an algorithm and the full vertex list at the interface. Open problems include minimum-cardinality rectangle partitioning.

XIV. Code Availability

The C++20 implementation is in `physdes-cpp` under the `luk036` organization. The tests build and run with `xmake build test_recti` and `xmake run test_recti`. The representation benchmark builds and runs with `xmake build bench_repr` and `xmake run bench_repr`; the convex-decomposition comparison uses `xmake build bench_cut` and `xmake run bench_cut`. The implementation provides `is_canonical`, `canonicalize`, `rpolygon_expand`, creation/canonicalization, the convex, explicit, and implicit phases, `rpolygon_cut_convex_full`, and the three-step rectangle pipeline.

[1] N. A. Sherwani, *Algorithms for VLSI physical design automation*, 3rd ed. Boston, MA: Kluwer Academic Publishers, 1999.

[2] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI physical design: From graph partitioning to timing closure*. Dordrecht: Springer, 2011.

[3] S. M. Sait and H. Youssef, *VLSI physical design automation: Theory and practice*. Singapore: World Scientific, 1999.

[4] F. P. Preparata and M. I. Shamos, *Computational geometry: An introduction*. New York: Springer, 1985. doi: 10.1007/978-1-4612-1098-6.

[5] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational geometry: Algorithms and applications*, 3rd ed. Berlin: Springer, 2008. doi: 10.1007/978-3-540-77974-2.

[6] J. M. Keil, "Polygon decomposition," in *Handbook of computational geometry*, J.-R. Sack and J. Urrutia, Eds., Amsterdam: Elsevier, 2000, pp. 491–518. doi: 10.1016/B978-044482537-7/50012-7.

[7] A. B. Kahng, C.-H. Park, X. Xu, and H. Yao, "Layout decomposition for double patterning lithography," in *Proc. IEEE/ACM int. Conf. Computer-aided design (ICCAD)*, 2008, pp. 465–472. doi: 10.1109/ICCAD.2008.4681616.

[8] L. W. Liebmann, "Layout impact of resolution enhancement techniques: Impediment or opportunity?" in *Proc. Int. Symp. Physical design (ISPD)*, 2003, pp. 110–117. doi: 10.1145/640000.640026.

[9] T. Ohtsuki, "Minimum dissection of rectilinear regions," in *Proc. IEEE international symposium on circuits and systems (ISCAS)*, Rome, Italy, 1982, pp. 1210–1213.

[10] Jr. Lipski Witold, "An $O(n \log n)$ Manhattan path algorithm," *Information Processing Letters*, vol. 19, no. 2, pp. 99–102, 1984, doi: 10.1016/0020-0190(84)90105-4.

[11] S. Nahar and S. Sahni, "Fast algorithm for polygon decomposition," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 7, no. 4, pp. 473–483, 1988, doi: 10.1109/43.3182.

[12] V. G. Soltan and A. Gorpinevich, "Minimum dissection of a rectilinear polygon with arbitrary holes into rectangles," *Discrete and Computational Geometry*, vol. 9, no. 1, pp. 57–79, 1993, doi: 10.1007/BF02189307.

[13] J. M. Keil, "Decomposing a polygon into simpler components," *SIAM Journal on Computing*, vol. 14, no. 4, pp. 799–817, 1985, doi: 10.1137/0214056.

[14] A. Lingas, "The power of non-rectilinear holes," in *Automata, languages and programming (ICALP)*, in Lecture notes in computer science, vol. 140. Berlin: Springer, 1982, pp. 369–383. doi: 10.1007/BFb0012784.

[15] J. K. Ousterhout, "Corner stitching: A data-structuring technique for VLSI layout tools," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 3, no. 1, pp. 87–100, 1984, doi: 10.1109/TCAD.1984.1270061.

[16] Calma Company, *GDS II stream format manual*, 6.0. 1987.

[17] SEMI, *Specification for OASIS—open artwork system interchange standard*. SEMI, 2003.

[18] W. R. Franklin and S. V. G. Magalhães, "Minimal representations of polygons and polyhedra," in *Spatial gems, volume 1*, ACM, 2022, pp. 33–48. doi: 10.1145/3548732.3548738.

[19] H. Edelsbrunner and E. P. Mücke, "Simulation of simplicity: A technique to cope with degenerate cases in geometric algorithms," *ACM Trans. Graph.*, vol. 9, no. 1, pp. 66–104, 1990.

[20] M. C. Couto, C. C. de Souza, and P. J. de Rezende, "AGPLIB: Art gallery problem instance library." https://www.ic.unicamp.br/~cid/Problem-instances/Art-Gallery/AGPVG/.